

CS-317 OPERATING SYSTEMS

Lecture 22



File management

Book 1 Chapter 12

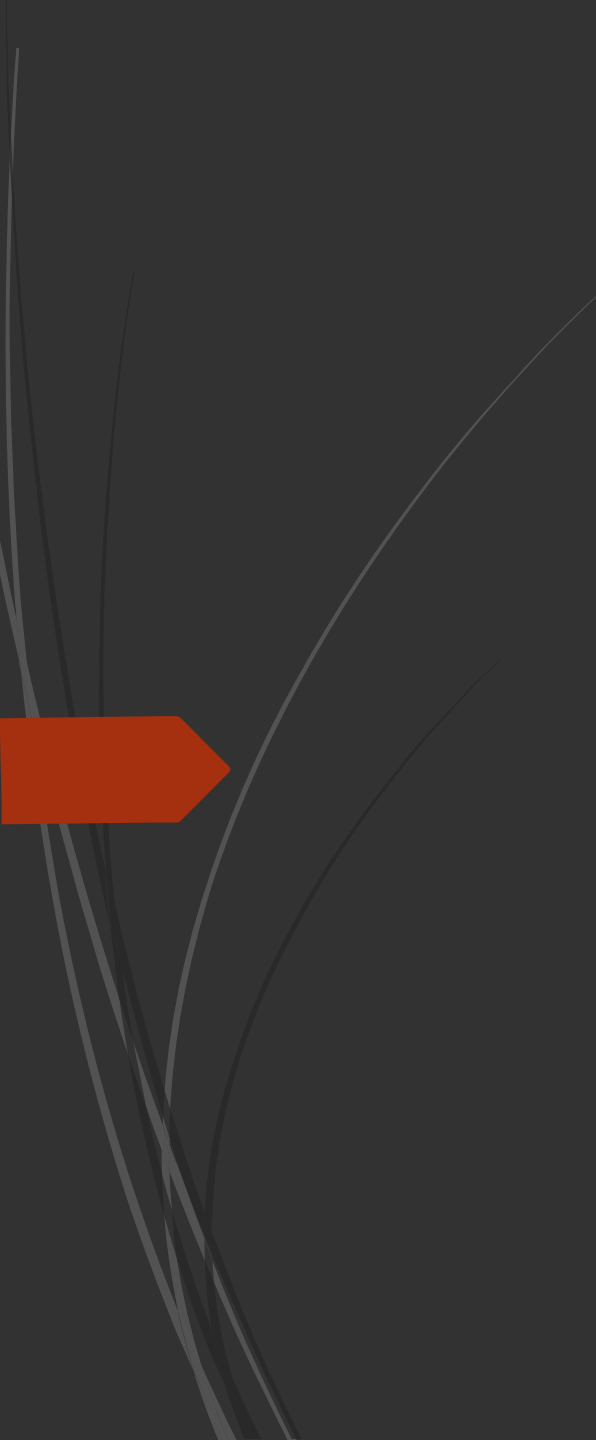




In the last lecture...

- We discussed functions on files.
- We explored file structure.
- We discussed operations on files.
- We understood file management system and explored its objectives, architecture, functions and performance issues.
- We discussed file organization.
- We explored different types of file organization.





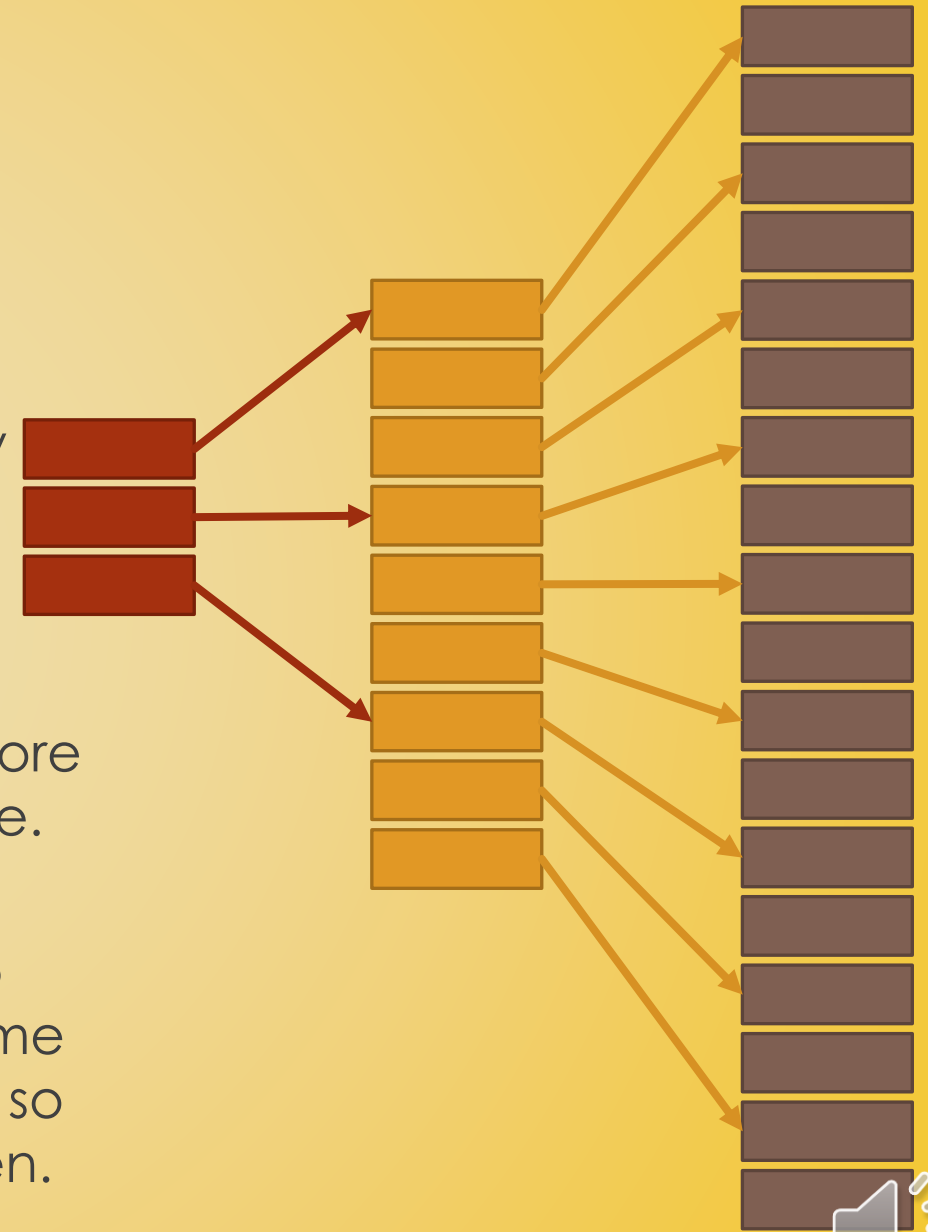
B-Trees

Data structures for file management

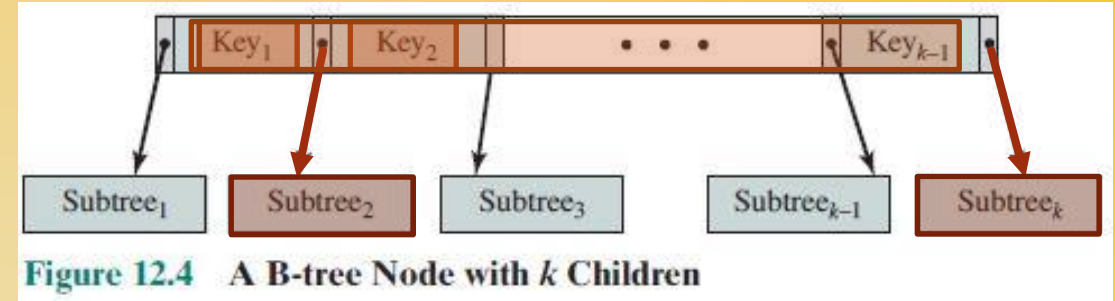


The need for B-trees

- For a large file or database, a single sequential file ordered by the primary key does not provide for rapid access.
- The simplest structure is a two-level organization.
- This structure can then be extended to more than two levels, resulting in a tree structure.
- Unless some discipline is imposed on the construction of the tree index, it is likely to end up with an uneven structure, with some short branches and some long branches, so that the time to search the index is uneven.



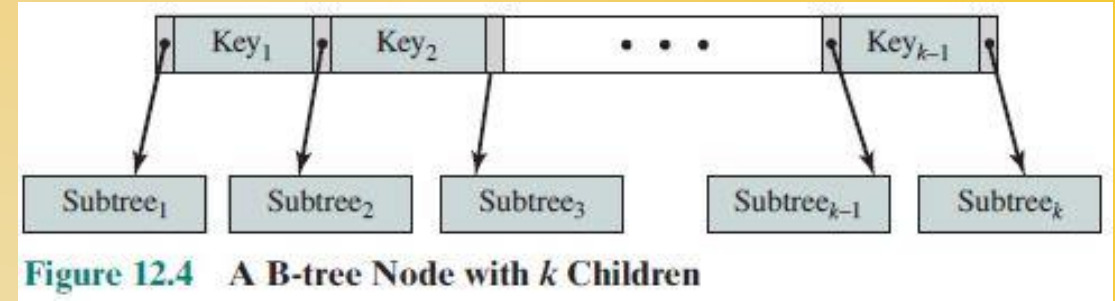
Formal definition



- A B-tree is a tree structure with the following characteristics:
 - The tree consists of a number of nodes and leaves.
 - Each node contains at least one key which uniquely identifies a file record, and more than one pointer to child nodes or leaves. The number of keys and pointers contained in a node may vary.
 - The maximum number of keys is same for all nodes.
 - The keys in a node are stored in increasing order. Each pointer points to the root of a subtree containing all nodes with keys greater than the key on the pointer's left but less than or equal to key on the pointer's right. The rightmost pointer points to the root for a subtree containing all keys greater than any keys in the node. Thus, each node has one more pointer than keys.



Mathematical definition



- A B-tree is characterized by its minimum degree d and satisfies the following properties:
 - Every node has at most $2d - 1$ keys and $2d$ children or pointers.
 - The root has at least 1 key and 2 children.
 - Every node, except for the root, has at least $d - 1$ keys and d pointers. As a result, each internal node, except the root, is at least half full and has at least d children.
 - All leaves appear on the same level and contain no information. This can also be implemented such that each bottom-level node contains null pointers.
 - A non-leaf node with k pointers contains $k - 1$ keys.



Relationship among keys

- Typically, a B-tree has a relatively large branching factor (large number of children).
- This results in a tree of low height.
- The upper level has $k - 1$ keys and k pointers and satisfies the following relationship:

$$\text{Key}_1 < \text{Key}_2 < \dots < \text{Key}_{k-1}$$

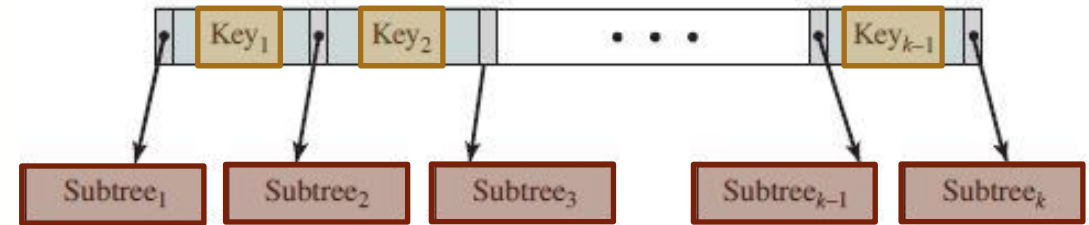
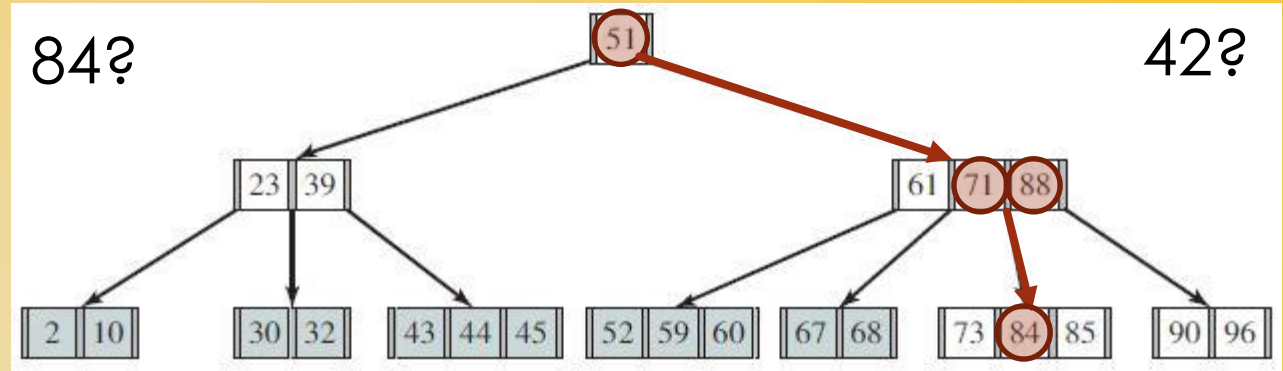


Figure 12.4 A B-tree Node with k Children

All the keys in Subtree ₁	are less than Key ₁
All the keys in Subtree ₂	are greater than Key ₁ and are less than Key ₂
All the keys in Subtree ₃	are greater than Key ₂ and are less than Key ₃
⋮	⋮
All the keys in Subtree _{$k-1$}	are greater than Key _{$k-2$} and are less than Key _{$k-1$}
All the keys in Subtree _{k}	are greater than Key _{$k-1$}



Searching in a B-tree



- To search for a key, you start at the root node.
- If the key you want is in the node, you're done.
- If not, you go down one level.
- There are three cases:
 - The key you want is less than the smallest key in this node. Take the leftmost pointer down to the next level.
 - The key you want is greater than the largest key in this node. Take the rightmost pointer down to the next level.
 - The key you want is between the values of two adjacent keys in this node. Take the pointer between these keys down to the next level.



Advantages of B-trees

- An advantage of this tree structure over other tree structures is that it is broad and shallow, so that the search terminates quickly.
- Furthermore, because it is balanced (all branches from root to leaf are of equal length), all searches are of equal length.



Insertion in a B-tree

Before and after each insertion, the B-tree must maintain balance. Suppose we want to insert a key k .

1. Search the tree for k . If k is not in the tree, then you have reached a node at the lowest level. Let this node be n .
2. If n has fewer than $2d - 1$ keys, i.e. it has space, then insert the key k into n in the proper sequence.
3. If n is full (having $2d - 1$ keys), then split n around its median key into two new nodes with $d - 1$ keys each and promote the median key m to the next higher level in a parent node p . If k is less than m , insert it into the left new node; otherwise insert it into the right new node. The original node n has been split into two nodes, one with $d - 1$ keys and one with d keys.

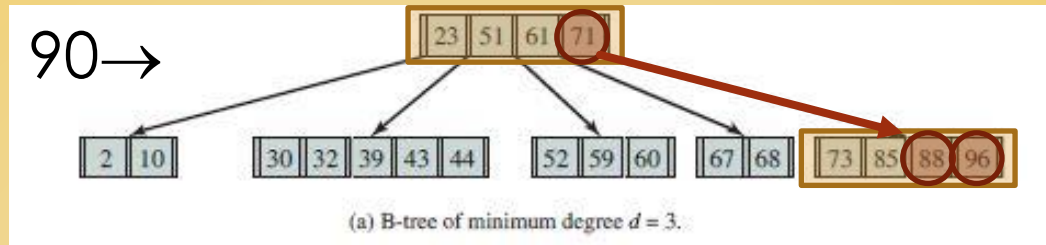


Insertion in a B-tree

Before and after each insertion, the B-tree must maintain balanced. This is done as follows:

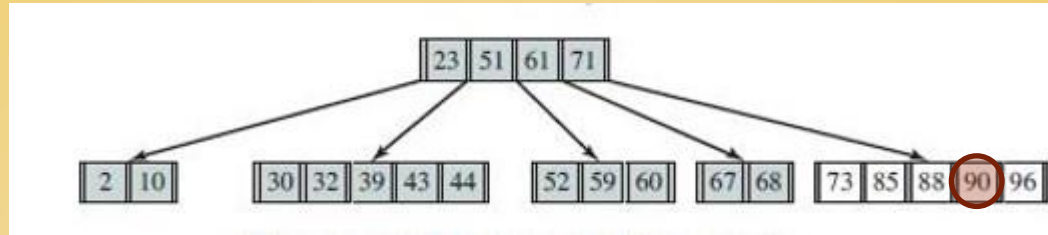
4. The median key **m** is inserted into a parent node **p** following the rules of step 3. Therefore, if **p** has space, **m** is inserted in it such that all keys of **p** are in increasing order. If **p** is already full, it must be split and its median key promoted to the next higher level.
5. If the process of promotion reaches the root node and the root node is already full, then insertion again follows the rules of step 3. However, in this case the median key becomes a new root node and the height of the tree increases by 1.





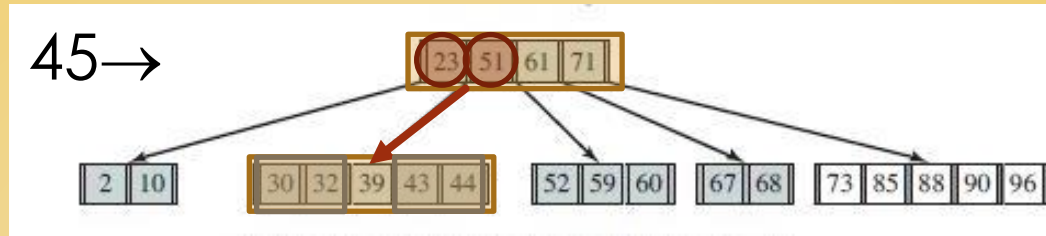
- Consider the tree shown here. $d = 3$.
- A node can have a maximum of $2d = 6$ pointers and $2d - 1 = 5$ keys.
- We want to insert key = 90.
- We start at the root.
- $90 > 71$.
- So we follow the rightmost pointer of the root downwards.
- The node has space.
- The right position for 90 is between 88 and 96.





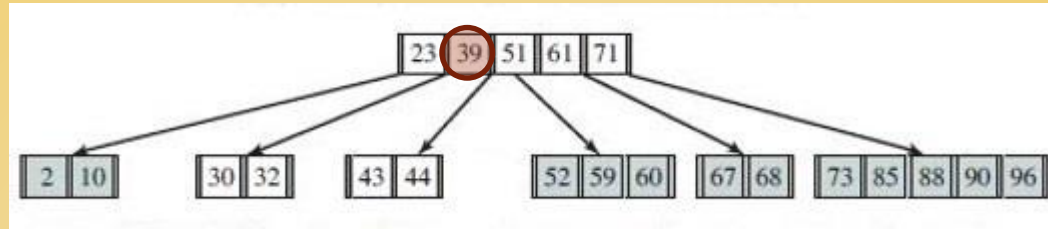
- Consider the tree shown here. $d = 3$.
- A node can have a maximum of $2d = 6$ pointers and $2d - 1 = 5$ keys.
- We want to insert key = 90.
- We start at the root.
- $90 > 71$.
- So we follow the rightmost pointer of the root downwards.
- The node has space.
- The right position for 90 is between 88 and 96.
- Finally, 90 is inserted in the node.





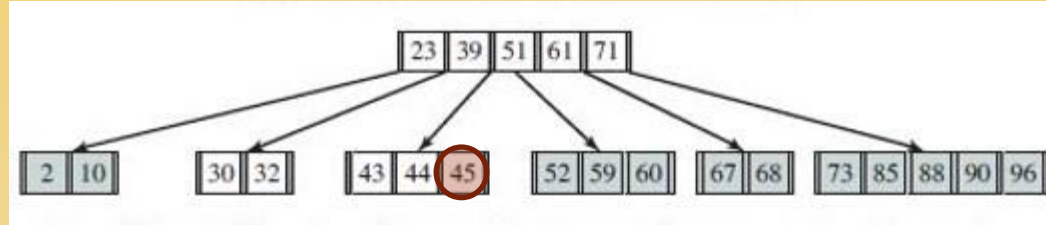
- We want to insert key = 45.
- We start at the root.
- $23 < 45 < 51$.
- So we follow the pointer of the root between 23 and 51 downwards.
- The node does not have space.
- It is broken into two nodes: the left node has 30, 32, the right node has 43, 44.





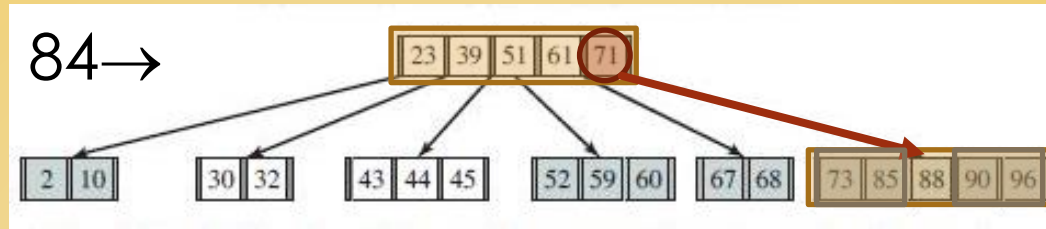
- We want to insert key = 45.
- We start at the root.
- $23 < 45 < 51$.
- So we follow the pointer of the root between 23 and 51 downwards.
- The node does not have space.
- It is broken into two nodes: the left node has 30, 32, the right node has 43, 44.
- The median node 39 is promoted up between 23 and 51.





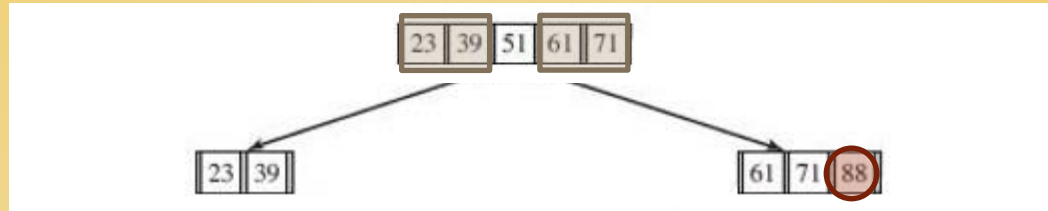
- We want to insert key = 45.
- We start at the root.
- $23 < 45 < 51$.
- So we follow the pointer of the root between 23 and 51 downwards.
- The node does not have space.
- It is broken into two nodes: the left node has 30, 32, the right node has 43, 44.
- The median node 39 is promoted up between 23 and 51.
- The right position for 45 is after 44 in the right new node.





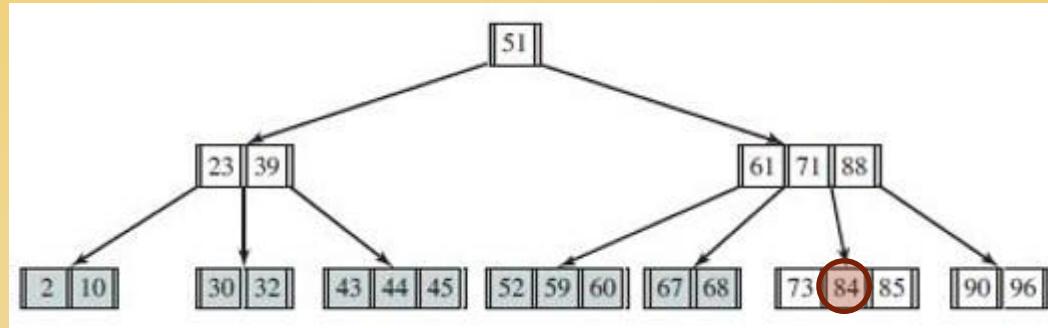
- We want to insert key = 84.
- We start at the root.
- $84 > 71$.
- So we follow the rightmost pointer of the root downwards.
- The node does not have space.
- It is broken into two nodes: the left node has 73, 85, the right node has 90, 96.
- The median node 88 is to be promoted up to the right of 71, but there is no space.





- The root is broken into two nodes: the left node has 23, 39, the right node has 61, 71.
- The median node 51 is promoted up.
- Now the node 88, which was to be inserted in the old root, is inserted after 71.





- The root is broken into two nodes: the left node has 23, 39, the right node has 61, 71.
- The median node 51 is promoted up.
- Now the node 88, which was to be inserted in the old root, is inserted after 71.
- The tree has increased in height.
- The right position for 84 is between 73 and 85 in the left new node.
- Finally, 84 is inserted in the node.



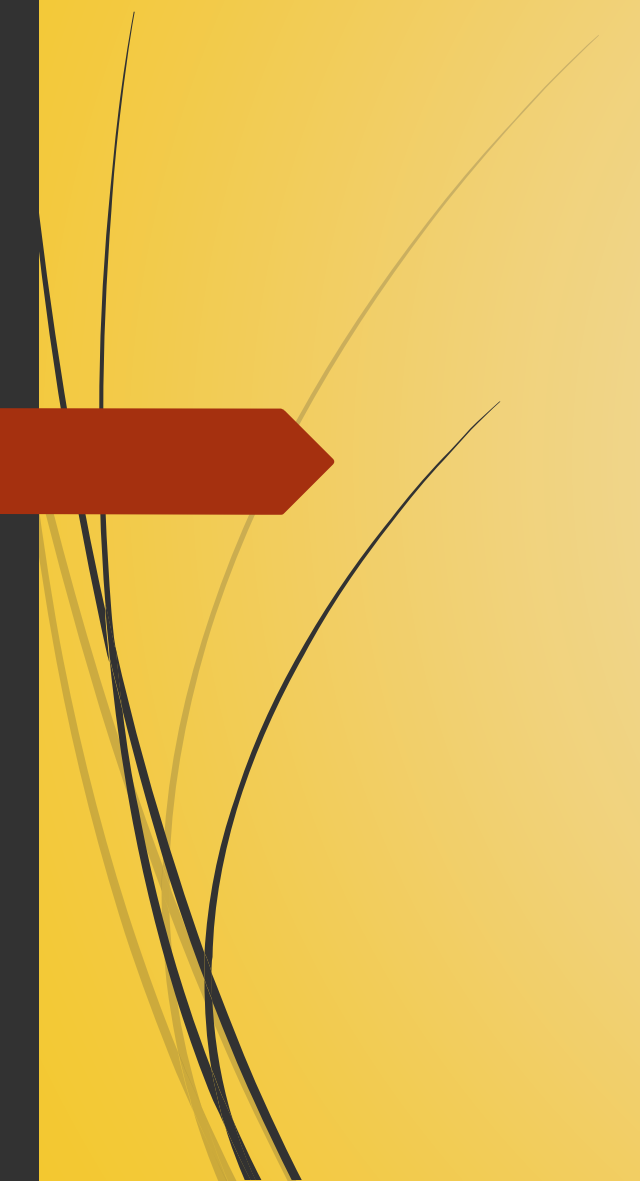
Homework:

Start with a node containing the first key only. Then keep on inserting keys in the order of the question. Use $d=3$. Then redo the exercise with $d=5$.

Construct a B-tree, adding these values to it:

65, 23, 12, 24, 59, 36, 4,
18, 33, 42, 21, 57, 65,
38, 20, 50, 11, 17





File Directories

Implementation and Types



Contents

- The directory contains information about the files, including:
 - Attributes,
 - Location, and
 - Ownership.
- The directory is itself a file, accessible by various file management routines.
- Although some of the information in directories is available to users and applications, this is generally provided indirectly by system routines.



Table 12.2 Information Elements of a File Directory

Basic Information	
File Name	Name as chosen by creator (user or program). Must be unique within a specific directory
File Type	For example: text, binary, load module, etc.
File Organization	For systems that support different organizations
Address Information	
Volume	Indicates device on which file is stored
Starting Address	Starting physical address on secondary storage (e.g., cylinder, track, and block number on disk)
Size Used	Current size of the file in bytes, words, or blocks
Size Allocated	The maximum size of the file
Access Control Information	
Owner	User who is assigned control of this file. The owner may be able to grant/deny access to other users and to change these privileges.
Access Information	A simple version of this element would include the user's name and password for each authorized user.
Permitted Actions	Controls reading, writing, executing, and transmitting over a network
Usage Information	
Date Created	When file was first placed in directory
Identity of Creator	Usually but not necessarily the current owner
Date Last Read Access	Date of the last time a record was read
Identity of Last Reader	User who did the reading
Date Last Modified	Date of the last update, insertion, or deletion
Identity of Last Modifier	User who did the modifying
Date of Last Backup	Date of the last time the file was backed up on another storage medium
Current Usage	Information about current activity on the file, such as process or processes that have the file open, whether it is locked by a process, and whether the file has been updated in main memory but not yet on disk



Operations on a directory

- **Search:** When an application references a file, the directory must be searched to find the entry corresponding to that file.
- **Create file:** When a new file is created, an entry must be added to the directory.
- **Delete file:** When a file is deleted, an entry must be removed from the directory.
- **List directory:** All or a portion of the directory may be requested. This request results in a listing of all files owned by the requesting user, plus some of the attributes of each file (e.g., type, access control information, usage information).
- **Update directory:** If a file attribute is stored in the directory, changing it requires a change in the corresponding directory entry.



Structure

- Some directory information may be stored in a header record associated with the file.
- This reduces directory size.
- A small directory can be kept in entirety in main memory, which improves speed.
- The simplest form of structure for a directory is that of a list of entries, one for each file.
- This structure could be kept in a simple sequential file, with the name of the file serving as the key.
- In some earlier single-user systems, this technique has been used.



Analysis of the sequential file implementation of a directory

- The simple list is not suited to supporting directory operations.
- Single user system:
 - The user may have many types of files, which he may like to organize in some way. The sequential directory provides no help in organizing the files.
 - The sequential directory forces the user to be careful not to use the same name for two different files.
- Shared system:
 - Unique naming becomes a serious problem.
 - It is difficult to conceal portions of the overall directory from users, so everyone can view everyone else's files.

In a shared system, the directory is usually visible to all users, so it is hard to hide or restrict access



Two-level scheme for directories

- There is one directory for each user, and a master directory.
- The master directory has an entry for each user directory, providing address and access control information.
- Each user directory is a simple list of the files of that user.
- This arrangement means that:
 - Names must be unique only within the collection of files of a single user.
 - The file system can enforce access restriction on user directories.
- It provides users no help in structuring collections of files.



Tree structure for directories

- A more powerful and flexible approach, and one that is almost universally adopted, is the hierarchical, or tree-structure, approach.
- There is a master directory, which has in it a number of user directories.
- Each of these user directories, in turn, may have subdirectories and files as entries.
- This is true at any level: at any level, a directory may consist of entries for subdirectories and/or entries for files.

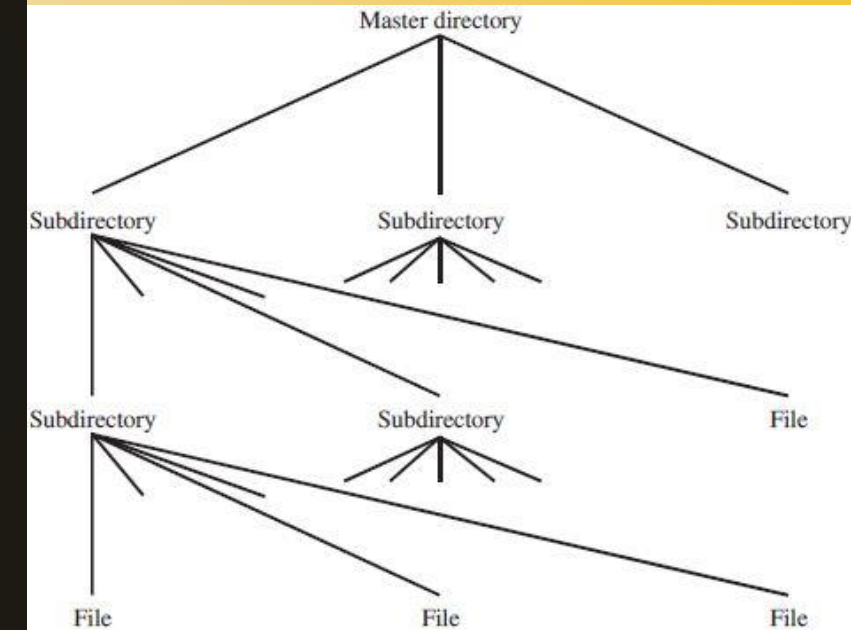


Figure 12.6 Tree-Structured Directory





Directory organization

- ➡ The simplest approach is to store each directory as a sequential file.
- ➡ When directories contain a very large number of entries, sequential organization leads to unnecessarily long search times.
- ➡ In that case, a hashed structure is preferred.





Naming

- Users must be able to refer to a file by a symbolic name.
- Each file in the system must have a unique name so that file references are unambiguous. But it is a burden on users to require they provide unique names, especially in a shared system.
- The use of a tree-structured directory minimizes the difficulty in assigning unique names.
- Any file in the system can be located by following a path down from the root directory until the file is reached.
- The series of directory names, culminating in the file name itself, constitutes a pathname for the file.
- If two files have the same names, but their paths from the root are different, their pathnames become different.



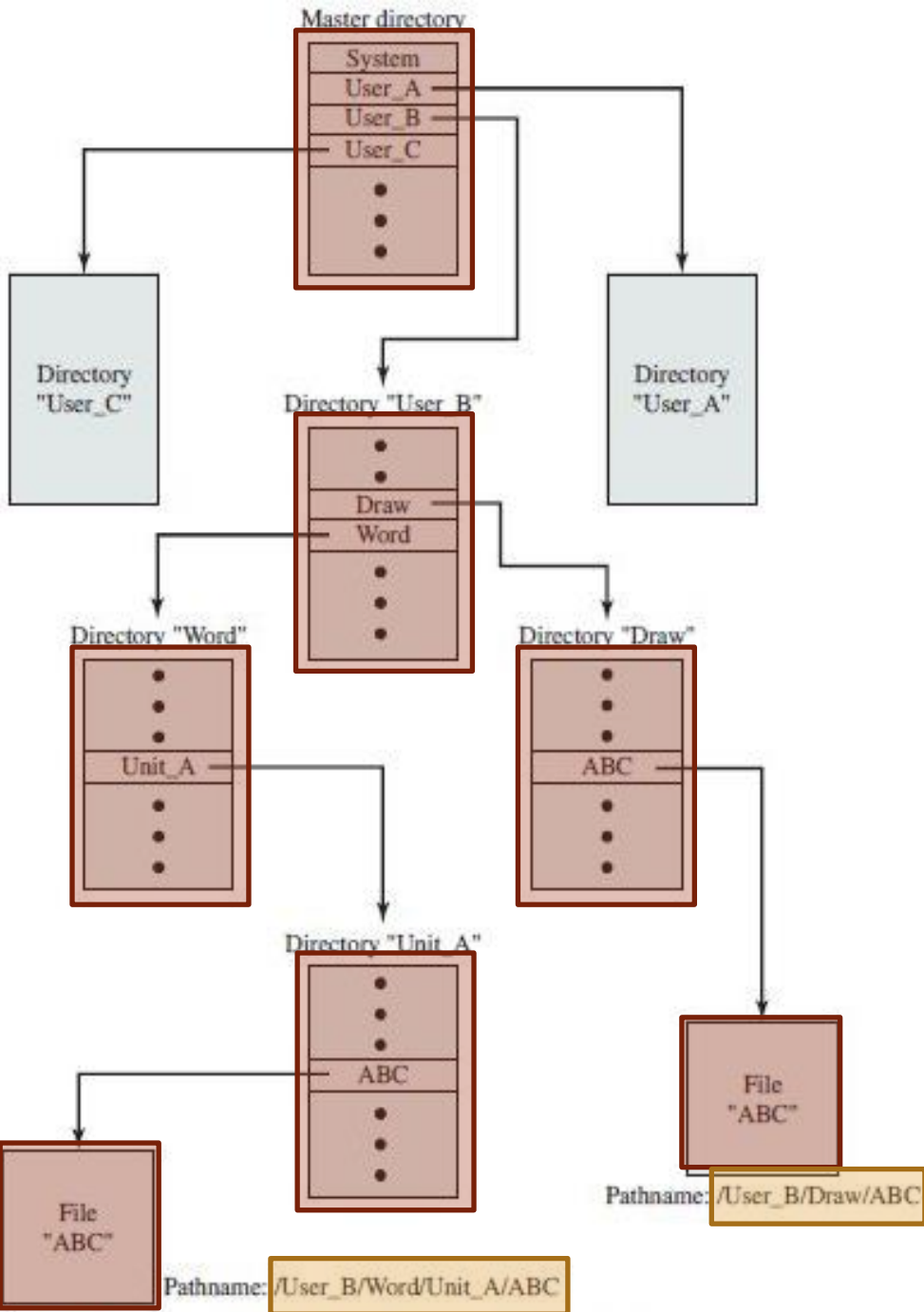
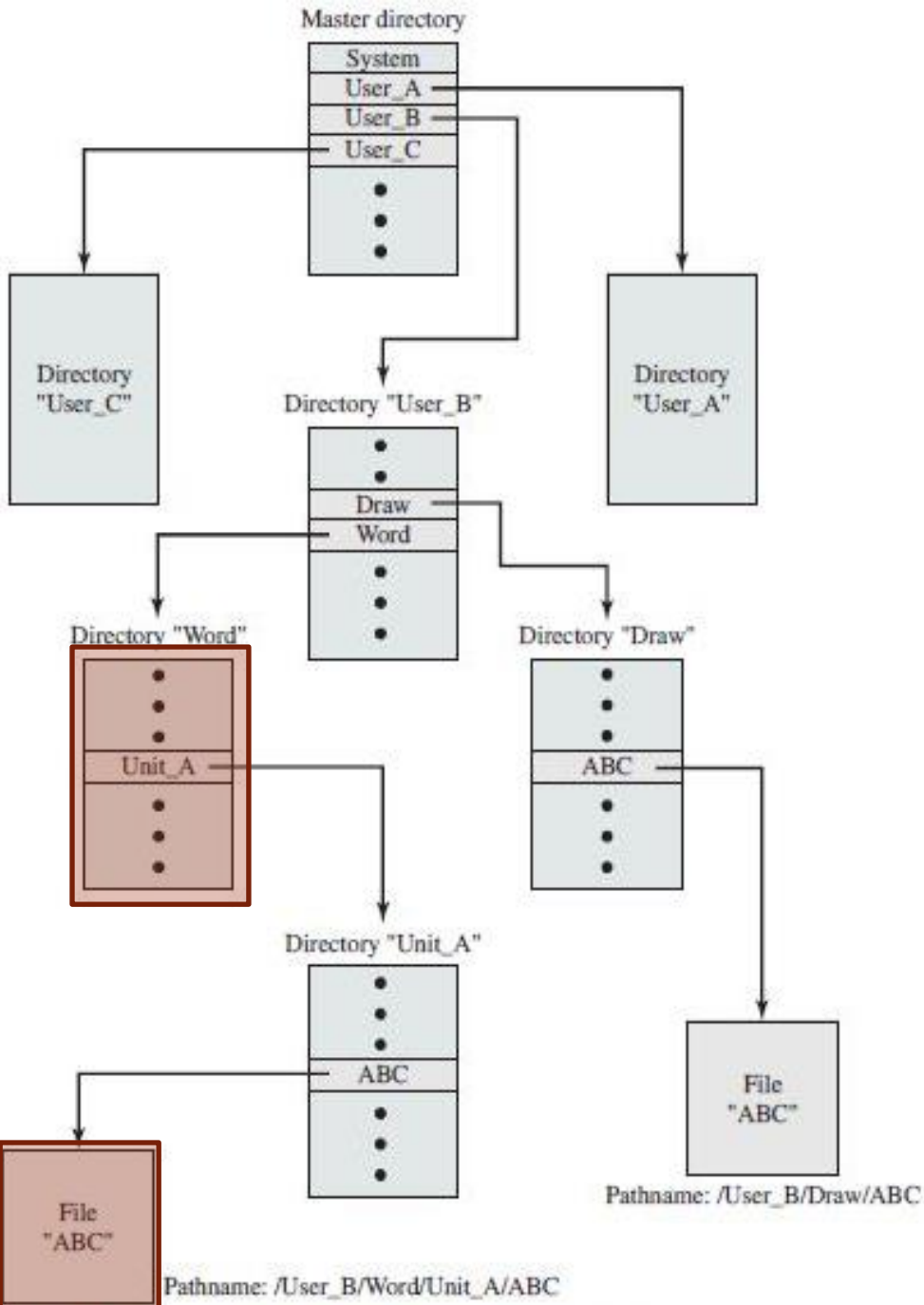


Figure 12.7 Example of Tree-Structured Directory

Pathnames

- The slash is used to delimit names in the sequence.
- The name of the master directory is implicit, because all paths start at that directory.
- It is perfectly acceptable to have several files with the same file name, as long as they have unique pathnames, which is equivalent to saying that the same file name may be used in different directories.





Working directory

- The user must give the entire pathname for every file reference.
- An interactive user or a process has a current directory, often referred to as the **working directory**.
- Files are referenced relative to the working directory.
- If the working directory for user B is **"Word,"** then the file can be sufficiently identified by **Unit_A/ABC**.



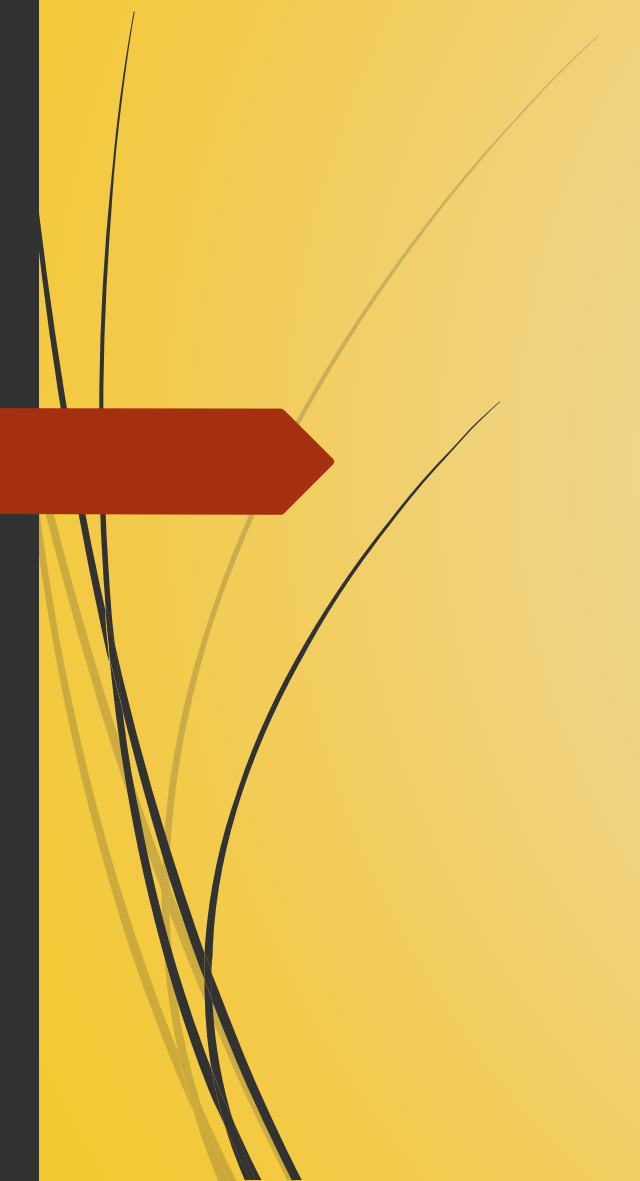
When a user logs in or a new process starts, the system automatically sets the working directory to the user's home directory.

This means all file operations (like creating or accessing files) will start from that default location unless the user changes it.

- When an interactive user logs on, or when a process is created, the default for the working directory is the user's home directory.
- During execution, the user can navigate up or down in the tree to change to a different working directory.

Home directory





File Sharing

Access rights and simultaneous access





Issues in multi-user systems

- In a multiuser system, there is almost always a requirement for allowing files to be shared among a number of users.
- Two issues arise:
 - Access rights, and
 - The management of simultaneous access.



Access rights

- The file system should provide a flexible tool for allowing extensive file sharing among users. Users or groups of users are granted certain access rights to a file.
- The following access rights can be assigned to a user for a file:
 1. None
 2. Knowledge
 3. Execution
 4. Reading
 5. Appending
 6. Updating
 7. Changing protection
 8. Deletion



Access rights

1. **None:** The user does not even know of the file, much less access it. To enforce this restriction, the user is prohibited from reading the directory that includes this file.
2. **Knowledge:** The user knows that the file exists and who its owner is. The user is able to request the owner for additional access rights.
3. **Execution:** The user can load and execute a program but cannot copy it.
4. **Reading:** The user can read the file, copy it and execute it. Some systems are able to enforce a distinction between reading and copying. In this case, the contents of the file can be displayed to the user, but the user cannot make a copy.



Access rights

5. **Appending:** The user can add data to the file, often only at the end, but cannot modify or delete any of the file's contents. This right is useful in collecting data from a number of sources.
6. **Updating:** The user can modify, delete, and add to the file's data. This normally includes writing the file initially, rewriting it completely or in part, and removing all or a portion of the data.
7. **Changing protection:** The user can change the access rights granted to other users. This right is held only by the owner of the file. In some systems, the owner can extend this right to others. To prevent abuse of this mechanism, the owner is able to specify which rights can be changed by the holder of this right.
8. **Deletion:** The user can delete the file from the file system.



Hierarchy of access rights

- The access rights discussed constitute a hierarchy, with each right implying those that precede it.
- Usually the person who initially created a file is its owner.
- The owner has all access rights and may grant rights to others.
- Access can be provided to different classes of users:
 - **Specific user:** Individual users who are designated by user ID.
 - **User groups:** A set of users who are not individually defined.
 - **All:** All users who have access to this system.

doc request

google cloud id



Simultaneous access

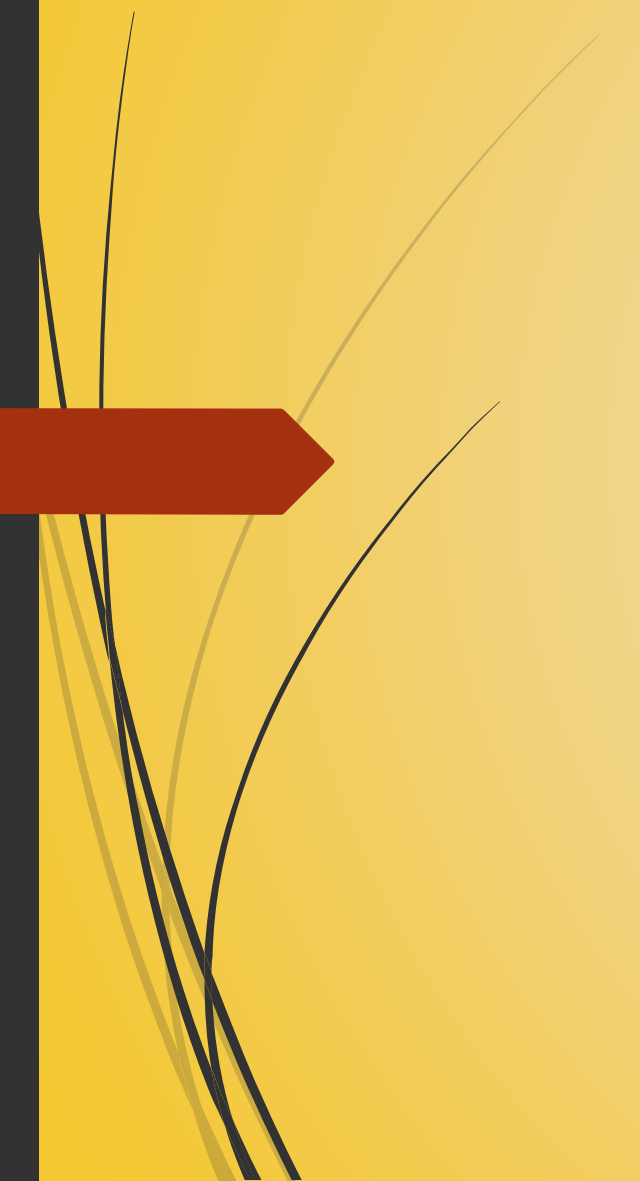
➡ When appending or updating access is granted on a file to more than one users, the operating system or file management system must enforce discipline.

➡ A **brute-force** approach is to allow a user to lock the entire file when it is to be updated.

➡ A **finer grain** of control is to lock individual records during update. This allows multiple users to update different records in the same file simultaneously.

➡ Essentially, this is the **readers / writers problem**. Issues of mutual exclusion and deadlock must be addressed in designing the shared access capability.





Record Blocking

Concept and techniques



Issues to consider

- Records are the logical unit of a structured file.
- Blocks are the unit of I/O with secondary storage.
- For I/O to be performed, records must be organized as blocks.
- There are several issues to consider.
 1. Should blocks be of fixed or variable length?
 2. What should the relative size of a block be compared to the average record size?



Fixed or variable sized blocks

- On most systems, blocks are of fixed length.
- This simplifies:
 - I/O,
 - Buffer allocation in main memory, and
 - The organization of blocks on secondary storage.



Size of a block

The larger the block, the more records that are passed in one I/O operation.

If a file is being searched sequentially, large block size is an advantage, because the number of I/O operations is reduced, speeding up processing.

because more records are read in one disk access, so fewer I/O operations are needed.

If records are being accessed randomly and no particular locality of reference is observed, larger blocks result in the unnecessary transfer of unused records.

Larger blocks require larger I/O buffers, making buffer management difficult.

If access is random, a large block size is inefficient because many extra records in the block are loaded but not used. This wastes I/O and memory.



Methods of blocking

Given the size of a block, there are three methods of blocking:

1. **Fixed blocking:** Fixed-length records are used, and an integral number of records are stored in a block. There may be unused space at the end of each block. This is referred to as internal fragmentation.

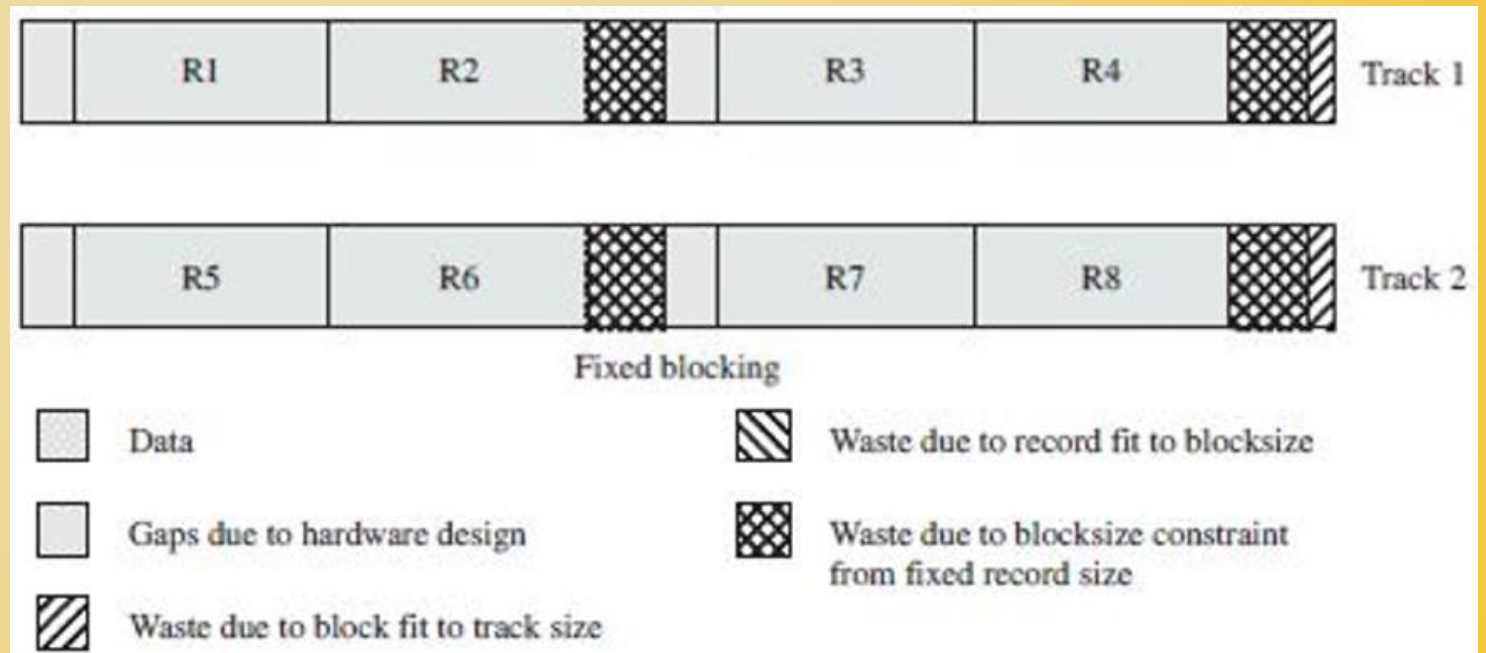


Figure 12.8 Record Blocking Methods [WIED87]



Methods of blocking

Given the size of a block, there are three methods of blocking:

2. **Variable-length spanned blocking:** Variable-length records are used and are packed into blocks with no unused space. Thus, some records must span two blocks, with the continuation indicated by a pointer to the successor block.

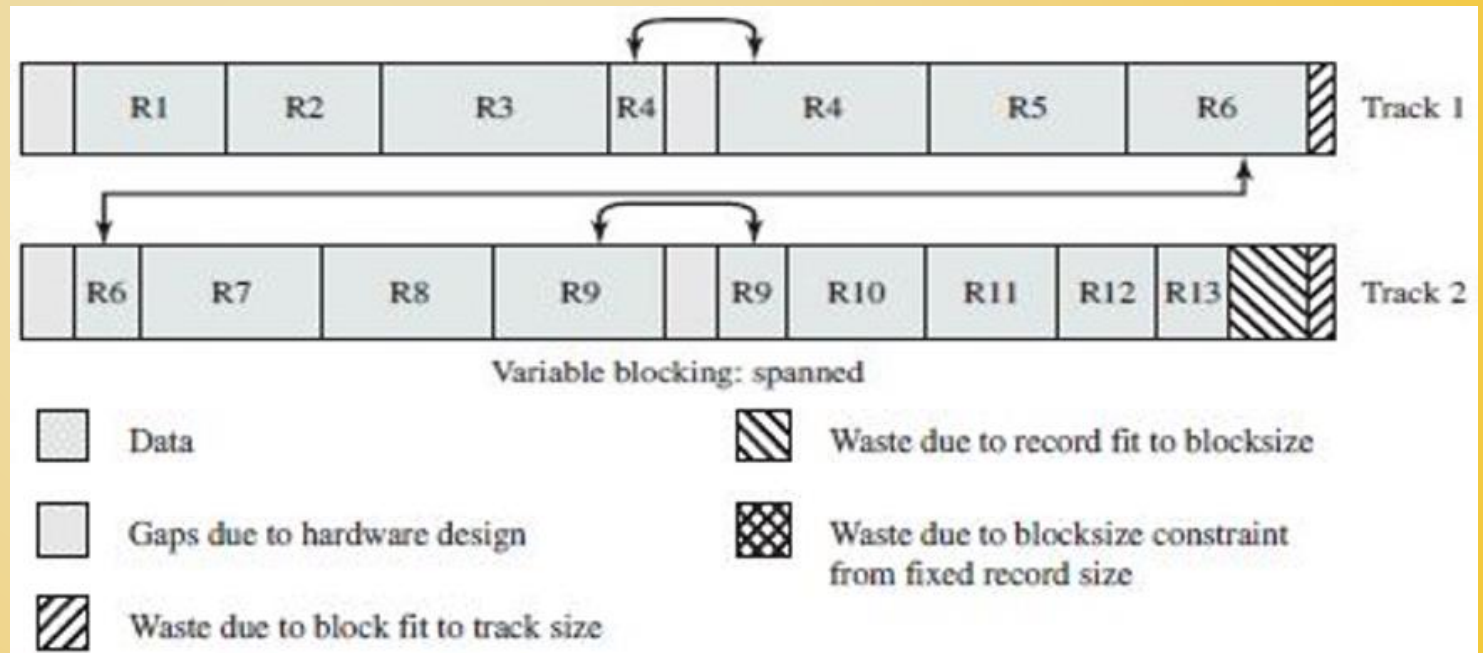


Figure 12.8 Record Blocking Methods [WIED87]

blocks are fixed but records can have different sizes



Methods of blocking

Given the size of a block, there are three methods of blocking:

3. **Variable-length unspanned blocking:** Variable-length records are used, without spanning. There is **wasted space in most blocks** because we can't use **the remainder of a block** if the next record is **larger than the remaining unused space**.

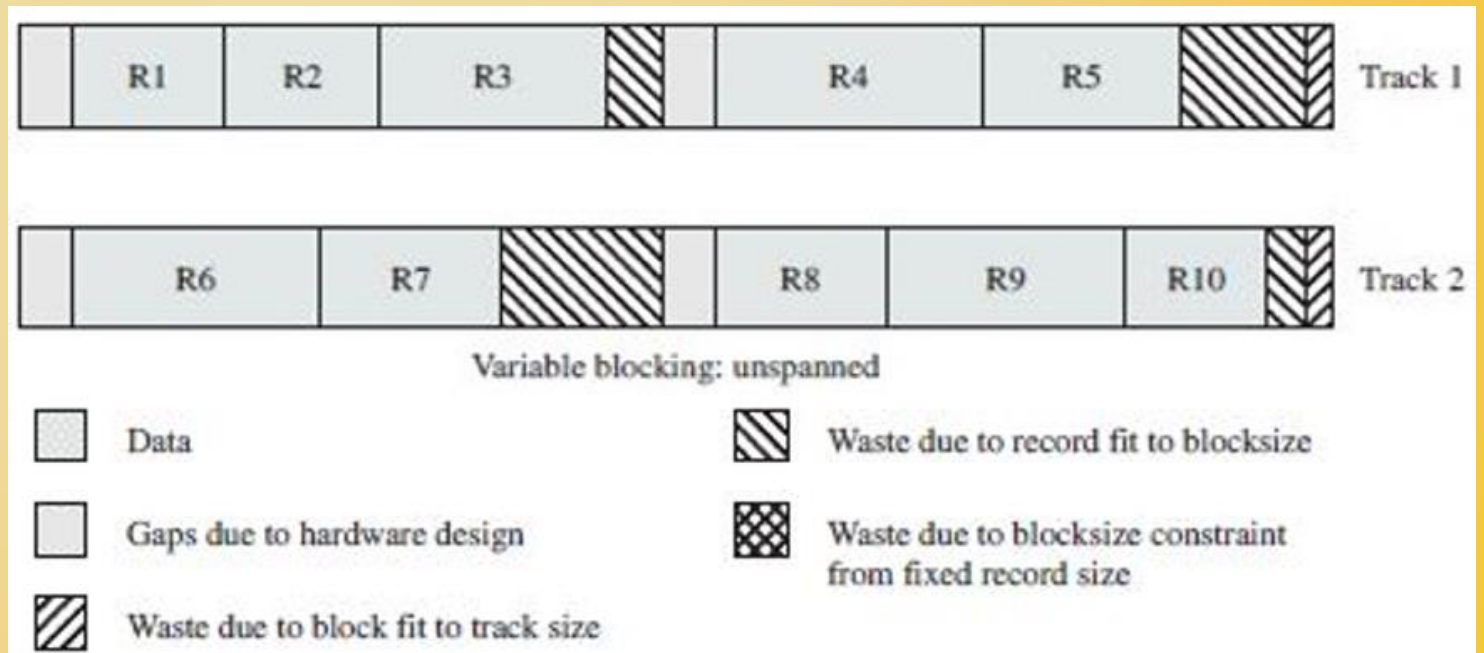


Figure 12.8 Record Blocking Methods [WIED87]



Analysis of various blocking choices

- Fixed blocking is the common mode for sequential files with fixed-length records.
- Variable-length spanned blocking is efficient for storage and does not limit the size of records. However, this technique is difficult to implement.
- Records that span two blocks require two I/O operations, and files are difficult to update, regardless of the organization.
- Variable-length unspanned blocking results in wasted space and limits record size to the size of a block.



Blocking choices in virtual memory

- In a virtual memory environment, it is desirable to make the page the basic unit of transfer.
- Pages are generally quite small.
- It is impractical to treat a page as a block for unspanned blocking.
- Accordingly, some systems combine multiple pages to create a larger block for file I/O purposes.
- This approach is used for VSAM files on IBM mainframes.



What did we learn?

In virtual memory, a **page** is the basic unit moved between disk and memory. Since pages are small, using **unspanned blocking** would waste space and be inefficient. So some systems group multiple pages into a larger block for file I/O, allowing more efficient disk transfers.

In short: multiple small pages are combined into larger blocks to improve I/O efficiency (e.g., VSAM).

- We discussed B-trees.
- We explored file directories.
- We discussed file sharing.
- We understood record blocking.



Things to do

